

Лекция 3: Основы программирования на Pascal: операторы, выражения, ввод и вывод данных

План лекции:

1. **Оператор присваивания** (формат записи, правило совместимости типов, примеры)
2. **Арифметические выражения**
3. **Логические операции** (AND, OR, NOT, таблицы истинности)
4. **Структура программы**
5. **Ввод и вывод данных** (Read, ReadLn, Write, WriteLn, форматный вывод)

Оператор присваивания

Оператор присваивания является одним из основных операторов любого языка программирования.

Оператор присваивания предназначен для присваивания переменной значения выражения.

Формат оператора присваивания:

`<переменная>:=<выражение>`

В операторе присваивания слева от знака присваивания «:=» записывается переменная, а справа – выражение, численное значение которого присваивается этой переменной.

Выполнение оператора присваивания приведет к вычислению выражения и присваиванию значения выражения переменной.

Обязательным является требование, чтобы выражение и переменная были совместимы по присваиванию.

Например, оператор присваивания

`y:=x+h/2;`

означает, что переменной `y` присваивается число, полученное в результате вычисления выражения `x+h/2` при известных числовых значениях переменных `x` и `h`.

Следует обратить внимание, что всем переменным в правой части оператора присваивания ранее уже были присвоены значения предыдущими операторами, например,

`x:=0; h:=1;`

Рассмотрим пример

`t:=t+1;`

Здесь переменной `t` присваивается значение, которое до этого момента имела переменная `t`, плюс 1. Если переменная `t` имела до выполнения этого оператора значение равно 7, то после исполнения оператора `t:=t+1` значение переменной будет равно 8.

Арифметические выражения

Арифметические выражения строятся из арифметических констант, переменных, функций и операций над ними.

Правила записи и смысл арифметических выражений в Паскале почти не отличаются от обычной математической записи.

В арифметических выражениях используются следующие операции и соответствующие знаки, представленные в таблице 3.

Таблица 3 - Операции

Операция	Знак операции
Сложение	+
Вычитание	-
Умножение	*
Деление	/
Деление целых чисел	div
Остаток от деления целых чисел	mod

Операции +, -, *, / имеют очевидный смысл. Результатом операции div будет целое значение, равное целой части результата деления первого числа на второе. Результатом операции mod будет целое значение, равное остатку от деления (но не дробной части).

Примеры арифметических операций:

Var

a,b,c:integer;

Begin

a:=10; b:=3;

c:=a mod b; {c равно 1 – равно остатку, но не дробной части}

c:= a div b; {c равно 3 – равно целой части }

b:=a+b; {b равно числу 13}

End.

Арифметические операции выполняются в порядке старшинства: сначала вычисление функции, затем умножение, деление, сложение, вычитание.

Для того, чтобы правильно записать арифметические выражения, нужно соблюдать следующие правила:

1. **Запись должна быть линейной** : все символы пишутся в строку на одном уровне. Проставляются все знаки операций (нельзя пропускать знак умножения).
2. Не допускаются два следующих подряд знака операций (нельзя A+-B; можно A+(-B)).
3. Операции с более высоким приоритетом выполняются раньше операций с меньшим приоритетом. **Порядок убывания приоритетов:**
 1. Вычисление функций;
 2. Унарная операция смены знака (-);
 3. *, /, div, mod;
 4. +, -.
4. Несколько записанных подряд операций одинакового приоритета выполняются последовательно слева направо.

5. Часть выражения, заключенная в скобки, вычисляется в первую очередь. (Например, $(A+B) * (C-D)$ – умножение производится после сложения и вычитания.)
6. Если в числителе или в знаменателе не одно число и не одна функция, то их нужно брать в скобки.

Таблица 4 – Стандартные функции операций

Функция	Назначение	Тип аргумента	Тип функции
ABS(x)	Абсолютное значение (модуль)	Integer / Real	соответствует типу аргумента
SQR(x)	Возведение в квадрат	Integer / Real	соответствует типу аргумента
SQRT(x)	Квадратный корень	Integer / Real	Real
SIN(x)	Синус угла (x в радианах)	Integer / Real	Real
COS(x)	Косинус угла (x в радианах)	Integer / Real	Real
ARCTAN(x)	Арктангенс (результат в радианах)	Integer / Real	Real
EXP(x)	Экспонента (e^x)	Integer / Real	Real
LN(x)	Натуральный логарифм ($x > 0$)	Integer / Real	Real
PI	Число π (3.14159...)	без аргумента	Real
INT(x)	Целая часть числа (как вещественное)	Real	Real
FRAC(x)	Дробная часть числа	Real	Real
ROUND(x)	Округление до ближайшего целого	Real	LongInt
TRUNC(x)	Отсечение дробной части (к нулю)	Real	LongInt
ORD(x)	Порядковый номер значения	Char / Boolean / Integer / перечислимый	LongInt

Функция	Назначение	Тип аргумента	Тип функции
CHR(x)	Преобразование кода в символ	Byte	Char
PRED(x)	Предыдущее значение	порядковый тип	соответствует типу аргумента
SUCC(x)	Следующее значение	порядковый тип	соответствует типу аргумента
LENGTH(s)	Длина строки	String	Integer
POS(sub, s)	Поиск подстроки (позиция первого вхождения)	String, String	Byte
COPY(s, i, n)	Копирование подстроки	String, Integer, Integer	String
UPCASE(ch)	Преобразование символа в верхний регистр	Char	Char

Логические операции

Логические значения true и false, отношения :=, <>, >, >=, <, <= и логические переменные называются первичными логическими выражениями. Существуют специальные логические операции, с помощью которых из первичных выражений могут составляться более сложные конструкции. Наиболее употребительные три из них:

Логическое And - И;

Логическое Or - ИЛИ;

Логическое отрицание Not – НЕ.

Значение логического выражения, которое состоит из двух первичных выражений A и B, связанных логической операцией, определяется таблицами истинности.

Таблица 5 - Таблица истинности операции NOT

A	Not A
False	True
True	False

Таблица 6 Таблица истинности операций AND, OR

A	B	A and B	A or B
False	False	False	False
False	True	False	True

A	B	A and B	A or B
True	False	False	True
True	True	True	True

Операция Not (НЕ). Эта операция относится только к одному логическому выражению A (говорят унарная операция). Если первичное логическое выражение A имеет значение true, то выражение not A (читается не A) имеет значение false, если A – false, то not A – true .

Операция And (И). Логическое выражение A and B (читается: A и B) имеет значение true только в том случае, когда оба первичных логических выражения A и B имеют значение true . Во всех остальных случаях выражение A and B имеет значение false .

Операция Or (ИЛИ). Логическое выражение A or B (читается: A или B) имеет значение true, если значение true имеет хотя бы одно из первичных логических выражений. В единственном случае, когда и A, и B имеют значение false, выражение A or B имеет значение false .

Логические операции применяются к данным логического типа. Операции отношения :=, <, >, >=, <=, <= имеют более низкий приоритет, чем арифметические операции: сначала выполняются арифметические операции, а затем операции отношения.

Условие a+b не равно c+d на языке Турбо Паскаль записывается так: a+b<>c+d .

Структура программы. Ввод и вывод данных

Программа на языке Pascal состоит из строк. Синтаксически программа состоит из заголовка и блока.

Блок состоит из 2 частей:

- описательный (может отсутствовать);
- исполнительный (без него программа не имеет смысла).

В описательной части блока находятся разделы: описание констант и описание переменных.

Исполнительная часть блока начинается с begin (начало), далее следует операторы языка, разделителем является точка с запятой.

Завершает раздел зарезервированное слово «end» с точкой.

Begin – начало блока (исполнительного)

End – конец исполнительного блока

<объявление программной единицы>

{раздел описаний}

Begin (начало)

{раздел исполнительных операторов}

End (конец)

1. Объявление программной единицы – это заголовок программы, модуля, процедуры.

Program <имя>;

Unit < имя >;

Function < имя >;

Procedure < имя >;

2. Раздел описаний – в данном разделе должны содержаться описания всех имен, используемых в исполнительном разделе.

Ввод и вывод данных в Pascal

Процедуры ввода-вывода. Почти каждая программа должна общаться с пользователем, то есть выводить результаты своей работы на экран и запрашивать у пользователя информацию с клавиатуры. Для того чтобы это стало возможным, в Турбо-Паскале имеются специальные процедуры (то есть небольшие вспомогательные программы), называются они процедурами ввода-вывода. Для того чтобы заставить процедуру работать в нашей программе, нужно написать её имя, за которым в скобках, через запятую перечислить параметры, которые мы хотим ей передать. Для процедуры вывода информации на экран параметрами могут служить числа или текстовые сообщения, которые должна печатать наша программа на экран.

1. Операторы ввода

Read <строка вывода>

Read (a1, a2, ..., an) – переменные, которые последовательно получают вводимые значения В ОДНУ СТРОКУ.

ReadLn <список вывода>

ReadLn (a1, a2, ..., an) - переменные, которые последовательно получают вводимые значения В СТОЛБИК.

Между операторами существуют различия:

- оператор Read заканчивает свое действие, когда все переменные списка получают свое значение. Те данные, которые были введены через пробел до нажатия Enter, могут быть использованы следующим оператором Read или ReadLn;

- оператор ReadLn, присвоив значение своим переменным, остальные значения пропускает, т.е. следующим оператором Read или ReadLn они не используются.

В процедурах ввода значения переменных вводятся с клавиатуры после набора всей программы и запуска её на исполнение.

2. Операторы вывода

Для вывода данных на экран используются операторы Write и WriteLn.

write(p1,p2,... pn); –выводит на экран значения выражений p1,p2,... pn, количество которых (n) неограничено. Выражения могут быть числовые, строковые, символьные и логические. Под выражением будем понимать совокупность некоторых действий, применённых к переменным, константам или литералам, например: арифметические действия и математические функции для чисел, функции для обработки строк и отдельных символов, логические выражения и т.п. Возможен форматный вывод, т.е. явное указание того, сколько выделять позиций на экране для вывода значения. Пример для вещественных типов: write(r+s:10:5); – вывести значение выражения r+s с выделением для этого 10 позиций, из них 5–после запятой. Для других типов все несколько проще: write(p:10); – вывести значение выражения p, выделив

под это 10 позиций. Вывод на экран в любом случае производится по правому краю выделенного поля.

`Writeln(p1,p2,... pn);` – аналогично `write`, выводит значения `p1,p2,... pn`, после чего переводит курсор на новую строку. Смысл параметров – тот же, замечания о форматном выводе остаются в силе. Существует вариант `writeln;` (без параметров), что означает лишь перевод курсора на начало новой строки.

Пример. Программа просит пользователя ввести с клавиатуры два целых числа и печатает на экране их сумму:

```
program PrintSum; <название программы>
var <раздел описания переменных >
    a,b: integer; <описание переменных>
begin <начало программы>
    write('Введите два числа:'); <вывод на консоль>
    readln(a,b); <ввод данных>
    writeln('Сумма a и b равна ',a+b); <вывод на консоль>
    readln; <нужен для того, чтобы консоль не исчезла сразу же после
отображения результата программы>
end. <конец программы>
```

В данной лекции были рассмотрены основные операторы и конструкции языка Pascal, необходимые для написания простейших программ. Изучен оператор присваивания, его формат записи и правило совместимости типов. Рассмотрены арифметические выражения, включая операции `div` и `mod`, а также стандартные математические функции. Студенты ознакомлены с логическими операциями `And`, `Or`, `Not` и их таблицами истинности. Изучена структура программы на Pascal, включающая заголовок, разделы описаний (`const`, `var`) и исполнительную часть с операторными скобками `begin..end`. Рассмотрены операторы ввода `Read` и `ReadLn`, а также операторы вывода `Write` и `WriteLn` с форматным выводом вещественных чисел.

Вопросы для проверки:

Вопрос 1 – Что такое оператор присваивания? Как он записывается?

Вопрос 2 – Чем отличается оператор `Read` от `ReadLn`? Чем `Write` от `WriteLn`?

Вопрос 3 – Какие арифметические операции вы знаете? Назовите 3 любые стандартные функции.